PROYECTO DE LABORATORIO NO. 2

Modalidad: Grupal
Valor: 10 puntos
Lenguaje: C#

Descripción general:

El grupo desarrollará una aplicación de consola en C# que resuelva un problema real de gestión de información y conectividad, aplicando de forma integrada las estructuras de datos estudiadas durante la segunda parte del curso: tablas hash, grafos, conjuntos disjuntos (Union-Find) y tries (árboles de prefijos).

El objetivo no es solo que el programa funcione, sino que el grupo justifique de manera técnica la elección de cada estructura de datos según las operaciones más frecuentes del problema (búsqueda, inserción, eliminación, recorrido y conectividad). Para este proyecto en particular, es obligatorio implementar y aplicar una Tabla Hash, un Grafo (con recorridos BFS y DFS, y el algoritmo de Dijkstra), un conjunto de estructuras Union-Find y un Trie, según se detalla en los requisitos técnicos. Al tratarse de un proyecto grupal, el nivel de exigencia es mayor: cada integrante debe dominar al menos una de las estructuras obligatorias y ser capaz de explicarla y defenderla individualmente.

Temas propuestos:

Elija uno de los siguientes escenarios:

- **Red de transporte urbano:** administra las paradas y rutas de buses de la ciudad. Cada parada tiene un código único, un nombre y coordenadas. Cada ruta conecta dos paradas con un tiempo o costo de viaje asociado, y algunas rutas pueden operar en un solo sentido. El sistema permite registrar paradas y rutas, localizar una parada por su código, sugerir nombres de parada mientras el usuario escribe (autocompletado), calcular la ruta más rápida o más económica entre dos paradas, y verificar si dos paradas pertenecen a la misma zona conectada de la red antes de habilitar una ruta nueva entre ellas.
- **Red social universitaria:** almacena los perfiles de los usuarios y sus relaciones de amistad o seguimiento dentro de una plataforma estudiantil. Cada usuario tiene un nombre de usuario único, datos de perfil y un nivel de afinidad con cada contacto. El programa permite registrar usuarios, buscar un perfil por su nombre de usuario, sugerir nombres de usuario mientras se escribe en el buscador, calcular el camino de conexión más fuerte entre dos usuarios (por ejemplo, para recomendaciones de amistad) y agrupar a los usuarios en comunidades o círculos según qué tan conectados están entre sí.
- **Logística de envíos:** controla los envíos de una empresa de paquetería que opera entre distintas ciudades del país. Cada envío tiene un código de rastreo único y datos completos: remitente, destinatario, ciudad de origen y ciudad destino. Las ciudades están conectadas por rutas de transporte con un costo y tiempo asociados, y algunas rutas son de un solo sentido. El programa permite registrar envíos, localizar un envío por su código de rastreo, sugerir nombres de ciudad

mientras se escribe, calcular la ruta más económica o más rápida entre dos ciudades, y determinar si dos ciudades pertenecen a la misma región de distribución antes de asignar una nueva ruta.

Requisitos técnicos mínimos:

El proyecto debe cumplir con lo siguiente para ser válido:

1. **Estructuras obligatorias:** El programa debe implementar y utilizar correctamente una Tabla Hash, un Grafo, un conjunto de estructuras Union-Find y un Trie para resolver el escenario elegido. Cada estructura debe estar justificada según la operación que resuelve dentro del problema, y debe quedar claro qué integrante del grupo es responsable de cada una.
2. **Integración entre estructuras:** las cuatro estructuras deben trabajar de forma coordinada dentro de un mismo flujo (por ejemplo: el Trie sugiere un nombre o código, la Tabla Hash localiza el registro completo, el Grafo calcula la ruta o relación entre dos elementos, y el Union-Find valida la conectividad antes de ejecutar el algoritmo de Dijkstra). No se aceptan estructuras aisladas que no se comuniquen entre sí.
3. **Operaciones:** incluir las operaciones básicas de cada estructura (inserción, búsqueda, eliminación donde aplique, recorrido e impresión), además de los recorridos BFS y DFS y el algoritmo de Dijkstra sobre el grafo, las operaciones de unión y búsqueda con compresión de camino en el Union-Find, y la búsqueda por prefijo en el Trie.
4. **Análisis de complejidad:** el manual técnico debe incluir el análisis de complejidad (notación Big O) de las operaciones principales de cada estructura implementada, comparando su comportamiento frente a las alternativas nativas de .NET.
5. **Datos dinámicos:** los datos no deben estar quemados en el código. Deben cargarse por medio de archivos .csv, .txt o ingresarse manualmente por medio de un menú.
6. **Menú interactivo:** el programa debe ofrecer un menú de consola claro que permita ejecutar todas las operaciones indicadas para las cuatro estructuras.
7. **Implementación propia:** la funcionalidad del programa debe ser implementada en su totalidad por el grupo. No se permite utilizar clases o estructuras de colecciones nativas de .NET (por ejemplo, List<T>, Dictionary, HashSet, SortedSet o PriorityQueue<TElement, TPriority>) para resolver la lógica de la Tabla Hash, el Grafo, el Union-Find o el Trie. El uso de tipos nativos se limita a soporte auxiliar (lectura de archivos, manejo de cadenas, y el uso de Queue<T> únicamente como estructura de control para el orden de visita en el recorrido BFS), nunca para reemplazar la lógica interna de las estructuras solicitadas.

Entregables:

1. Código fuente en C#, debidamente comentado, indicando qué integrante del grupo desarrolló cada estructura.
2. Manual técnico: incluye diagrama de flujo, diagrama de clases, explicación de la implementación y análisis de complejidad (Big-O), justificando la elección de cada estructura de datos utilizada. Se entrega en formato PDF.
3. Manual de usuario: instrucciones claras del funcionamiento del programa, incluyendo el uso de cada opción del menú.

4. Demostración funcional y defensa individual del proyecto ante el docente, donde cada integrante debe defender, como mínimo, la estructura de la cual fue responsable, y responder preguntas sobre el resto del sistema.
5. Interfaz gráfica obligatoria

Consideraciones

- El proyecto se realizará en grupos, formados por los propios estudiantes.
- La solución debe compilar correctamente para tener derecho a calificación y/o revisión funcional.
- La entrega se realizará únicamente en el espacio habilitado en el portal, con un solo envío por grupo.
- Durante la revisión podrán solicitarse cambios de funcionalidad a cualquier integrante, para comprobar el dominio individual de todo el grupo sobre el proyecto.

La copia parcial o total entre grupos tendrá una calificación de cero puntos para todos los involucrados y se notificará a la coordinación correspondiente.

Política de uso de IA y dominio individual del proyecto

El uso de herramientas de Inteligencia Artificial (IA) está permitido únicamente como guía de apoyo o consulta. No sustituye el proceso de análisis, diseño ni programación que debe realizar cada integrante del grupo.

Todo lo que se presente en el proyecto (análisis, diagrama de flujo, diagrama de clases, código y decisiones implementadas) debe poder ser explicado, justificado y defendido individualmente por el estudiante.

Durante la defensa presencial, cada estudiante deberá demostrar dominio del proyecto, explicando la lógica del programa, ejecutándolo correctamente y realizando modificaciones solicitadas por el docente.

Si durante la evaluación se evidencia que un estudiante:

- No comprende el código desarrollado,
- No puede explicar el análisis o las decisiones implementadas,
- No puede realizar modificaciones básicas solicitadas,
- Demuestra dependencia total de un compañero o de herramientas externas,

Esto será considerado incumplimiento de los objetivos formativos del curso y podrá calificarse como plagio o uso indebido de recursos, aplicándose una penalización individual de hasta el 100% de la nota del proyecto, según la gravedad del caso.

Universidad Rafael Landívar
Identidad Jesuita en Guatemala

Facultad de Ingeniería
Ingeniería en Informática y Sistemas
Área de Informática
Estructura de Datos II

Criterio	Descripción	Valor (%)	Excelente (100%)	Bueno (75%)	Regular (50%)	Deficiente (25%)	No se evidencia (0%)
1. Manejo de estructuras de datos	El programa implementa correctamente todas las funciones solicitadas: gestión de datos mediante Tabla Hash, Grafo (BFS, DFS, Dijkstra), Union-Find y Trie, integrados en un mismo flujo.	**50**	Cumple todos los requerimientos sin fallos	Cumple ≥75% de funcionalidades, con errores menores	Cumple ≥50% con errores que afectan usabilidad	Cumple solo ≥25% con muchas fallas y errores	Cumple menos del 30% de requerimientos y muestra errores al ejecutarse
2. Funcionalidades adicionales	Gestión correcta de la ubicación, existencia y carga de archivo con datos de entrada	**15**	Cumple todos los requerimientos sin fallos	Cumple ≥75% de funcionalidades, con errores menores	Cumple ≥50% con errores que afectan usabilidad	Cumple solo ≥25% con muchas fallas y errores	Cumple menos del 30% de requerimientos y muestra errores al ejecutarse
3. Diseño y codificación	Claridad, modularidad, uso de funciones nativas para la gestión de archivos y estructuras.	**10**	Código modular, claro y documentado. Solo usa funciones nativas para gestionar archivos y sus estructuras	Código modular, en su mayoría bien documentado. Uso adecuado de funciones nativas para cadenas y propias para estructuras, con errores menores	Código poco modular o confuso. Dificultades en uso de funciones nativas para archivos y sus estructuras	Código desordenado y sin modularidad. Documentación insuficiente o ausente. Uso inadecuado de funciones nativas y propias	No presenta código funcional o incumple requisitos de diseño
4. Documentación técnica y manual de usuario	Manual técnico con diagramas de flujo, clases y explicación de implementación, manual de usuario con instrucciones claras.	**5**	Documentación completa, incluye los diagramas, la documentación es clara y ordenada	Documentación casi completa con mínimos vacíos, falta alguno de los diagramas	Documentación parcial, estructura poco clara, diagramas incompletos	Documentación deficiente, faltan los diagramas	Omite la documentación técnica y de usuario

5. Evidencia de competencias CG2 – Liderazgo solidario y transformador	Trabajo colaborativo, coordinación y aporte equitativo.	**5**	Colaboración efectiva, roles claros y equitativos	Colaboración adecuada con pequeños desequilibrios	Colaboración desigual, no colabora de forma constante	Se evidencia una colaboración mínima	No se evidencia colaboración alguna
6. Evidencia de competencias CG4 – Gestión del conocimiento técnico-científico	Integración y aplicación de conceptos de análisis léxico, estructuras de datos y generación de reportes.	**10**	Explica y aplica fundamentos teóricos de forma apropiada en todo el proyecto	Aplica la mayoría de conceptos	Aplicación parcial de teoría	Se evidencia una aplicación mínima de la teoría	Falta comprensión y aplicación de los conceptos teóricos solicitados
7. Evidencia de competencias CE9 – Gestión estratégica de TI	Desarrollo de un prototipo funcional que organiza información y genera reportes para toma de decisiones.	**5**	Prototipo sólido con integración completa de funcionalidades	Prototipo funcional con fallas menores	Prototipo limitado o incompleto, con errores que afectan significativamente el uso o funcionamiento	Prototipo incompleto, con fallas y errores frecuentes	No se evidencia gestión adecuada de TI
8. Presentación y dominio individual	Defensa oral presencial: cada estudiante debe explicar, ejecutar y realizar cambios al código.	**-100**	**(-0)** Explica con claridad y demuestra dominio total	**(-10)** Explica bien con leves dificultades	**(-30)** Explicación incompleta, dominio parcial	**(-60)** Explicación deficiente, poco dominio del proyecto realizado	**(-90)** No demuestra comprensión de lo implementado